



Experiment: 8(a)

Student Name: Bipasha

UID: 23BCS10735

Branch: BE-CSE

Section/Group: EP-801-A

Semester: 6th

Date of Performance: 23-03-26

Subject Name: Competitive Coding-II

Subject Code: 23CSP-308

1. Aim:

To design and implement a dynamic programming algorithm that computes the length of the Longest Common Subsequence (LCS) between two given strings text1 and text2 using 2D DP.

2. Objective:

To design and implement a dynamic programming algorithm that computes the length of the Longest Common Subsequence (LCS) between two given strings text1 and text2 using a 2D DP table, in order to efficiently determine the maximum length of a sequence that appears in both strings while maintaining their relative order.

3. Algorithm:

1. **Start**
2. **Let:**
 - $m = \text{length of text1}$
 - $n = \text{length of text2}$
3. **Create a 2D array $dp[m+1][n+1]$**
4. **Initialize:**
 - Set all values of first row and first column to 0
 - ($dp[i][0] = 0$ and $dp[0][j] = 0$)
5. **Fill the DP table:**
 - For $i = 1$ to m
 - For $j = 1$ to n :



a. If characters match:

- If $\text{text1}[i-1] == \text{text2}[j-1]$
→ $\text{dp}[i][j] = 1 + \text{dp}[i-1][j-1]$

b. If characters do not match:

- $\text{dp}[i][j] = \max(\text{dp}[i-1][j], \text{dp}[i][j-1])$

6. Final Answer:

- LCS length = $\text{dp}[m][n]$

7. Stop

4. Implementation/Code:

```
#include <vector>
#include <string>
using namespace std;

class Solution {
public:
    int longestCommonSubsequence(string text1, string text2) {
        int m = text1.length();
        int n = text2.length();

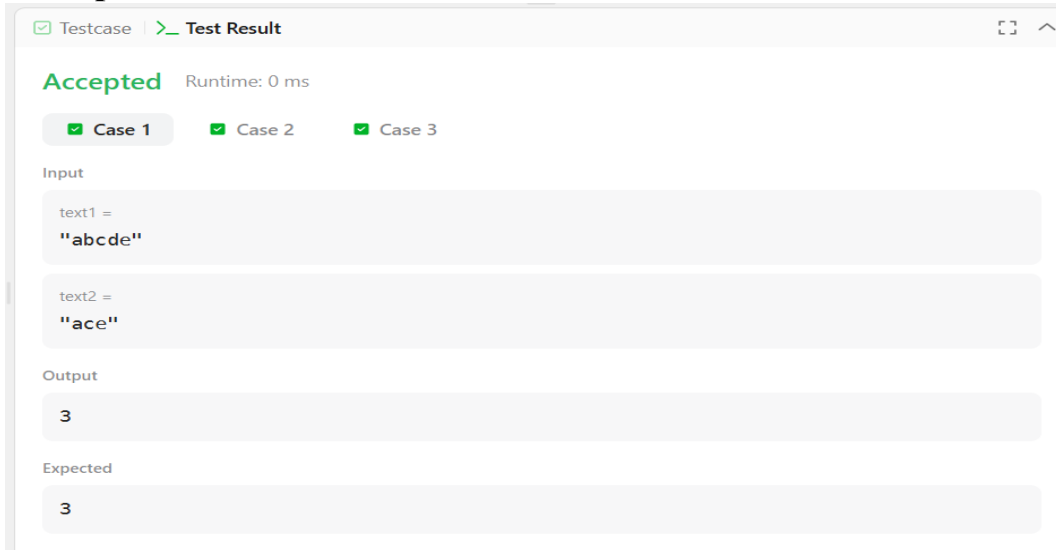
        vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1[i - 1] == text2[j - 1])
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        return dp[m][n];
    }
};
```



5. Output



6. Time Complexity: $O(m \times n)$

Space Complexity: $O(m \times n)$

7. Learning Outcome

1. Understand the concept of Longest Common Subsequence (LCS) and how it differs from substring problems.
2. Apply dynamic programming (DP) to break a complex problem into smaller subproblems.
3. Construct and use a 2D DP table to store intermediate results efficiently.
4. Develop bottom-up DP solutions and avoid redundant computations.
5. Analyze time and space complexity of DP algorithms.



Experiment: 8(b)

Student Name: Bipasha

UID: 23BCS10735

Branch: BE-CSE

Section/Group: EP-801-A

Semester: 6th

Date of Performance: 23-03-26

Subject Name: Competitive Coding-II

Subject Code: 23CSP-308

1. Aim:

To design and implement a dynamic programming algorithm that computes the minimum number of coins required to make a target amount using given coin denominations. If the amount cannot be formed, return -1.

2. Objective:

To design and implement a dynamic programming algorithm that determines the minimum number of coins required to make a given target amount using specified coin denominations, and to return -1 if the amount cannot be formed, ensuring an efficient and optimal solution.

3. Algorithm:

Step 1: **Start**

Step 2: Let n = amount

Step 3: Create an array $dp[0...n]$

Step 4: **Initialize:**

- $dp[0] = 0$
- For $i = 1$ to n , set $dp[i] = n + 1$ (represents infinity)

Step 5: **Compute values:**

For $i = 1$ to n :

- For each coin in $coins[]$:
 - If $coin \leq i$:



- $dp[i] = \min(dp[i], dp[i - \text{coin}] + 1)$

Step 6: **Check result:**

- If $dp[n] > n$:
 - Return -1
- Else:
 - Return $dp[n]$

Step 7: **Stop**

4. Implementation Code:

```
#include <vector>
using namespace std;

class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        vector<int> dp(amount + 1, amount + 1);

        dp[0] = 0;

        for (int i = 1; i <= amount; i++) {
            for (int coin : coins) {
                if (coin <= i) {
                    dp[i] = min(dp[i], dp[i - coin] + 1);
                }
            }
        }

        if (dp[amount] > amount)
            return -1;
        else
            return dp[amount];
    }
};
```



5. Output:

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

coins =
[1,2,5]

amount =
11

Output

3

Expected

3

- **Time Complexity:** $O(n \times \text{amount})$
- **Space Complexity:** $O(\text{amount})$

6. Learning Outcomes:

1. Understand the concept of dynamic programming and its use in optimization problems.
2. Apply 1D DP (bottom-up approach) to solve minimum cost / minimum count problems.
3. Identify and use optimal substructure and overlapping subproblems.
4. Design efficient algorithms with improved time complexity over recursion.
5. Handle edge cases such as when a solution is not possible (returning -1).



Experiment: 8(a)

Student Name: Bipasha

UID: 23BCS10735

Branch: BE-CSE

Section/Group: EP-801-A

Semester: 6th

Date of Performance: 23-03-26

Subject Name: Competitive Coding-II

Subject Code: 23CSP-308

1. Aim:

To design and implement a dynamic programming algorithm that computes the length of the Longest Common Subsequence (LCS) between two given strings text1 and text2 using 2D DP.

2. Objective:

To design and implement a dynamic programming algorithm that computes the length of the Longest Common Subsequence (LCS) between two given strings text1 and text2 using a 2D DP table, in order to efficiently determine the maximum length of a sequence that appears in both strings while maintaining their relative order.

3. Algorithm:

1. **Start**
2. **Let:**
 - $m = \text{length of text1}$
 - $n = \text{length of text2}$
3. **Create a 2D array $dp[m+1][n+1]$**
4. **Initialize:**
 - Set all values of first row and first column to 0
 - $(dp[i][0] = 0 \text{ and } dp[0][j] = 0)$
5. **Fill the DP table:**
 - For $i = 1$ to m For
 - $j = 1$ to n :



a. If characters match:

- If $\text{text1}[i-1] == \text{text2}[j-1]$
→ $\text{dp}[i][j] = 1 + \text{dp}[i-1][j-1]$

b. If characters do not match:

- $\text{dp}[i][j] = \max(\text{dp}[i-1][j], \text{dp}[i][j-1])$

6. Final Answer:

- LCS length = $\text{dp}[m][n]$

7. Stop

4. Implementation/Code:

```
#include <vector>
#include <string>
using namespace std;

class Solution {
public:
    int longestCommonSubsequence(string text1, string text2) {
        int m = text1.length();
        int n = text2.length();

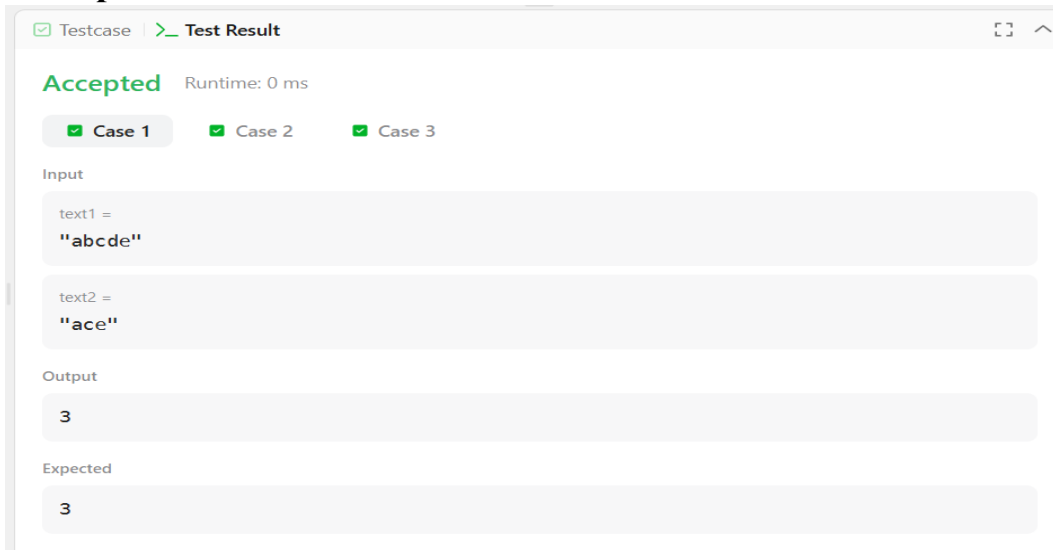
        vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1[i - 1] == text2[j - 1])
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        return dp[m][n];
    }
};
```




5. Output



6. Time Complexity: $O(m \times n)$
Space Complexity: $O(m \times n)$

7. Learning Outcome

1. Understand the concept of Longest Common Subsequence (LCS) and how it differs from substring problems.
2. Apply dynamic programming (DP) to break a complex problem into smaller subproblems.
3. Construct and use a 2D DP table to store intermediate results efficiently.
4. Develop bottom-up DP solutions and avoid redundant computations.
5. Analyze time and space complexity of DP algorithms.



Experiment: 8(b)

Student Name: Aditi Tyagi

UID: 23BCS10641

Branch: BE-CSE

Section/Group: EP-801-A

Semester: 6th

Date of Performance: 23-03-26

Subject Name: Competitive Coding-II

Subject Code: 23CSP-308

1. Aim:

To design and implement a dynamic programming algorithm that computes the minimum number of coins required to make a target amount using given coin denominations. If the amount cannot be formed, return -1.

2. Objective:

To design and implement a dynamic programming algorithm that determines the minimum number of coins required to make a given target amount using specified coin denominations, and to return -1 if the amount cannot be formed, ensuring an efficient and optimal solution.

3. Algorithm:

Step 1: **Start**

Step 2: Let n = amount

Step 3: Create an array $dp[0...n]$

Step 4: **Initialize:**

- $dp[0] = 0$
- For $i = 1$ to n , set $dp[i] = n + 1$ (represents infinity)

Step 5: **Compute values:**

For $i = 1$ to n :

- For each coin in $coins[]$:
 - If $coin \leq i$:



- $dp[i] = \min(dp[i], dp[i - \text{coin}] + 1)$

Step 6: **Check result:**

- If $dp[n] > n$:
 - Return -1
- Else:
 - Return $dp[n]$

Step 7: **Stop**

4. Implementation Code:

```
#include <vector>
using namespace std;

class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        vector<int> dp(amount + 1, amount + 1);

        dp[0] = 0;

        for (int i = 1; i <= amount; i++) {
            for (int coin : coins) {
                if (coin <= i) {
                    dp[i] = min(dp[i], dp[i - coin] + 1);
                }
            }
        }

        if (dp[amount] > amount)
            return -1;
        else
            return dp[amount];
    }
};
```



5. Output:

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

coins =
[1,2,5]

amount =
11

Output

3

Expected

3

- **Time Complexity: $O(n \times \text{amount})$**
- **Space Complexity: $O(\text{amount})$**

6. Learning Outcomes:

1. Understand the concept of dynamic programming and its use in optimization problems.
2. Apply 1D DP (bottom-up approach) to solve minimum cost / minimum count problems.
3. Identify and use optimal substructure and overlapping subproblems.
4. Design efficient algorithms with improved time complexity over recursion.
5. Handle edge cases such as when a solution is not possible (returning -1).